Parallel Brutus: The First Distributed, FPGA Accelerated Chess Program *

Chrilly Donninger Alex Kure
Ulf Lorenz
University of Paderborn,
Faculty of Computer Science, Electrical Engineering and Mathematics
Fürstenallee 11, D-33102 Paderborn

Abstract

In times when the FPGA technology has reached maturity such that complex designs are possible, the boarder between hard- and software vanishes. It is now possible to develop fine grained parallel applications without the long-lasting chip design cycles. Simultanously, by the help of message passing libraries like MPI it is easy to write coarse grained parallel applications. The chess program Brutus is a high level application which profits from both worlds.

When playing board games like chess, checkers, othello etc., computers use game tree search algorithms to evaluate a position. The most spectacular success of a game playing program so far, has been the victory of the chess machine 'Deep Blue' vs. G. Kasparov, the best human chess player in the world, at that time. The world has now reached the point that a chess program can not only win one match against a human top player, but they are just getting stronger than the best human players.

We describe the design philosophy, general architechture and performance of the chess program Brutus, which has just won an International Grandmaster Tournament with seven wins and 4 draws. Brutus is split up in a hardware- and software engine. The hardware (indeed in FPGA simulated hardware) calculates the time critical part of the search tree near the leaves. The part near the root is done in software, where it is much easier to to develop sophisticated algorithms. The FPGA cards allows the implementation of sophisticated chess knowledge without decreasing the computational speed. Thus, Brutus follows its famous predecessors Belle and Deep Blue.

1 Introduction

The dream of a competitive chess machine goes back to the year 1769, when the Turk played in an Austrian court with a human player hiding inside and making the moves. With the invention of von Neumann machines, the idea to build a superior chess machine revived in the 1940s [15]. The early chess programs tried to mimic the human chess style. In the 1970s Chess 4.5 [16], however, demonstrated that emphasizing the search speed might be more fruitful. Belle[3] was a special hardwired chess machine which became a national master in the early 1980s. Cray Blitz [9] running on a Cray supercomputer, Hitech, another special purpose chess machine, and Deep Thought[8] were the top programs in the 1980s. Then, in 1992, for the first time a conventional PC program won the open Computer Chess World Championships. It was the ChessMachine by Ed Schroeder, a professional computer chess programmer. From that time on, nearly only PCs dominated the world of computer chess: ChessMachine, Fritz, Shredder etc. With one well known exception: In 1997, IBM's Deep Blue[7], developed by Feng-Hsiung Hsu and Murray Campbell, won the historical 6-game match against Garry Kasparov. That highlight is still of singular quality, although the playing strengths of present top programs seem to cross the borderline beyond the strongest human players.

For most of the interesting board games we do not know the correct values of all positions. Therefore, we are forced to base our decisions on heuristic or vague knowledge.

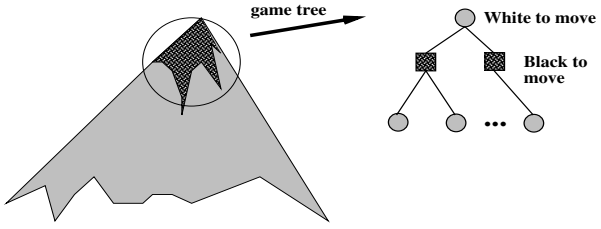

Figure 1. Only the (pre-)chosen partial tree is examined by a search algorithm.

* Sponsors: ChessBase GmbH, Alphadata Inc, Myricom Inc., Technical Support: Paderborn Center for Parallel Computing

0-7695-2132-0/04/$17.00 (C) 2004 IEEE

Typically, a game playing program consists of three parts: a move generator, which computes all possible chess moves in a given position; an evaluation procedure which implements a human expert's chess knowledge about the value of a given position (these values are quite heuristic, fuzzy and limited) and a search algorithm, which organizes a forecast:

At some level of branching, the allover game tree (the complete one, defined by the game) is cut, the artificial leaves of the resulting subtree are evaluated with the heuristic evaluations, and these values are propagated to the root of the game tree, as if they were real ones. The astonishing observation over the last 40 years in the chess game and some other games is: the game tree acts as an error filter. The faster and the more sophisticated the search algorithm, the better the search results!

Usually the tree to be searched is examined by the help of the Alphabeta algorithm [10] or the MTD(f)-algorithm [1]. In professional game playing programs, mostly the Negascout [13] variant of the Alphabeta algorithm is used.

The approximation of the real root value by the help of fixed-depth searches leads to good results. Nevertheless, several enhancements that form the search tree more individually have been found. Some of these techniques are domain independent like Singular Extensions [2], Nullmoves [4], Conspiracy Number Searches [12] [14], and Controlled Conspiracy Number Search [11]. Many others are domain dependent. The form of the search tree strongly determines the quality of the search result.

The search procedure, as well as the evaluation procedure and the move generator are mostly implemented in software on some standard PC. Nevertheless, it is also possible to encode the algorithms into a hardware design, which has two advantages: first, the procedures can be processed in a very few cycles such that the execution speed can significantly be increased. Second, on standard PCs a tradeoff between the search speed and the implementation of chess knowledge occurs. This tradeoff does not exist in a hardware design. Many evaluation features can be processed in parallel, which requires additional space, but no additional time. The FPGA technology yields further advantages. Debugging is easier and any improvements can be added instantly. Time-consuming manufacturing cycles do not exist, and therefore it additionally is less expensive to incorporate the changes.

In this paper, we discuss the design philosophy, general architecture and performance of Brutus, one of the strongest present chess programs in the world. Section two starts with a hardware overview about Brutus. In section 3, we discuss the software architecture by starting off with the FPGA part followed by the parallel search algorithm on the PC side. Last but not least, Section 4 deals with experimental results. We encounter playing results and speedup measurements.

2 System

Brutus is a pure engine which needs a graphical user interface (GUI). The GUI is the ChessBase/Fritz GUI, running on a PC with WindowsXP. It controls a program which is called 'plink' [**?**]. This connects the end-user PC via the Internet with our Linux cluster. Our cluster consists of 4 Dual PC server nodes, which have 2 CPUs, and a PCI controller, that is able to handle two PCI busses simultaneously. Each PCI bus can be supplied with one FPGA card. When the Brutus engine starts, 1,2,4 or 8[1] MPI processes are created. Each of these processes is mapped onto one of the processors and one of the FPGA cards is associated with it, as well. The server nodes themselves are interconnected by a Myrinet interconnection network, which is state of the art. The algorithm which we use are state-of-the-art scalable. Thus, in principle, the number of used processors is unlimited.

3 Software Architecture

As already stated, Brutus partially runs on PCs and partially on FPGA cards. This is especially true for the search algorithm. The reason is that the bottleneck of the system is the PCI bus. Therefore, on the one hand we cannot make the whole search run on the PC side. On the other hand, it is not clever to put all the search on the FPGA card, because it is much more difficult to develop a sophisticated search algorithm in hardware. This tradeoff is tuned such that the PCI bus is exactly before becoming a bottleneck, and we perform the last 3 plies of an n-ply search (a ply is half a move, the move of one player) on the FPGA side, inclusively the quiescence search and all evaluations. Such a depth-3 search is initiated about 100,000 times per second per processor. In this sense, the PCI-bus latency allows no smaller work packages for the card. We use Alphadata's asynchronous standard driver routines to control the bus. Asynchronous means that when the PC invokes an FPGA search, it sends a 32-bit word to the card and ponders for a 32-bit answer. While the PC ponders, it manages e.g. MPI messages.

The most significant differences between Deep Blue (as far as Deep Blue features have been published) and Brutus are:

- The Deep Blue full-position evaluator has a systolic architecture. The board is scanned and evaluated line by line. As a consequence, the full evaluation takes 11 cycles. In Brutus, the evaluation over the whole board is calculated at once, and the full evaluation takes 3 cycles. In order to reduce processing time, Deep Blue

[1] target architecture in Nov.'03, at present only 4 FPGA cards available

0-7695-2132-0/04/$17.00 (C) 2004 IEEE

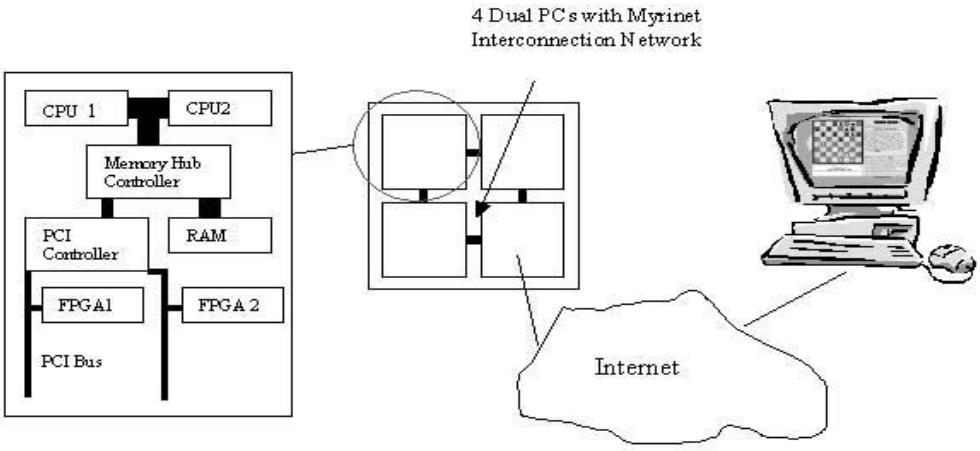

Figure 2. The System Architecture: 4 Dual Pentium with 2 FPGA cards per node are interconnected with an and-user PC via the Internet.

therefore used two evaluation stages. The 'fast evaluator' in the first stage is nothing else than a first guess, based on the material balance. If the material balance is too far away from the current search bound of the position, the full evaluation is skipped. "For example, if either side has lost a queen while we're expecting the position to be even, clearly there's no reason for a complete evaluation" [7]. This approach has its problems. If we set the margin for the skip too high (e.g. 1 queen), there are almost no time savings. If we set the margin to 1 pawn, we get considerable time savings. In this case, however, we assume, that the full positional evaluation cannot become greater than 1 pawn. Modern chess is rather dynamic and it is not uncommon, that one side sacrifices a minor piece for a positional advantage. The Deep Blue team implemented a number of additional tricks to circumvent this problem, but according to Hsu some evaluation parts like the king's safety and king-attack points were much too low. This is the consequence of the slow full evaluator. Brutus' spectacular style (see e.g. the game against Romanishin, Lippstadt Round 1) is only possible because we developed a 3 cycle full evaluator.

- For the efficiency of the Alphabeta search algorithm, a good move ordering is extremely important. In its hardware search Deep Blue uses a static, simple approach. Moves are ordered according the piece value of the square that it moves. In the case of a tie (or if the square is empty), the moves were ordered according their square numbers. Because of this ordering, the search over and over captures a defended knight with the queen. Brutus' search uses additional, dynamic information for move ordering. The dynamic ordering learns from search experience, that the queen-capture (in the just mentioned example) is bad and proposes to try some other moves first. The Brutus FPGA-search implements also the "null-move", the most powerful pruning technique beside Alphabeta cutoffs.

  Some further design decisions are different. E.g. in Deep Blue the material value of 1 pawn is 128 points. The value of a pawn is the basic evaluation unit. We knew from former experience, that such a fine grained evaluation is of no use. There are no positional concepts which have a precision of 1/128 of a pawn. Such a fine-grained evaluation would simply produce white noise, which is dominated by some higher evaluation terms. In Brutus, a pawn has therefore a value of only 64 points. This makes the evaluation adder tree more compact and faster.

3.1 The FPGA

Brutus' FPGA software is written in the Verilog hardware description language. In Verilog, the 'program' is mainly structured in so called modules. Each of these modules has an interface to other modules as well as a description of its contents. A module can be viewed as a black box with certain input bits, output bits, and a functional logic inside. A module starts with 'module <Modulname>' and ends with 'endmodule'. If a statement within a module is

written without parentheses, it represents pseudo-code for a group of operations. '<Modulename>();' is the instantiation of a module, which is similar to an object instantiation in a C++ program. Nevertheless, it is not the same, as all modules act in parallel.

At the present, we use a XilinX based VirtexV1000E card from Alphadata. This card was one of the first which are large enough to place a high level chess program on it. 'Start with the biggest and fastest chip you can get today and wait till the prices come down' [17].

We use 66 out of 96 Block-RAMs, 9,019 of 12,288 Slices, 5,302 of 12,544 TBUFS, 534 of 24,576 Flip-Flops, and 17,022 of 24,576 LUTs. We can run the FPGA with 30MHz, the longest path consists of 53 logic levels.

There are 4 basic operations: Make/Unmake a move on the board (1 cycle), generate a move ($2 \cdot 2$ cycles). Generating a move is a two step procedure. In the first 2 cycles the to-square is calculated, in the next 2 cycles the from-square. Evaluating a position takes 3 cycles.

These operations are performed partially overlapped, and also some speculative computing is done. E.g. during a calculation of a from-square, the next to-square is calculated simultaneously. It depends on the position, if this result can be used. An upper bound for the number of cycles per node is 9 cycles. This results in 3.2 million search nodes per second. Assuming 20 percent loss because of latency, we come up with 2.5 million nodes per second per FPGA card. It is difficult to estimate how our program would behave as a pure software solution. Our fpga-simulator is about 1000 times slower, such that we gain a factor of 10, considering that a PC can be clocked with 3 GHz at the moment. Nevertheless, you can implement some ideas different in software. We estimate that we gain a factor of 3, comparing a Virtex I 1000 chip with a 3.0 GHz Intel processor.

As the ratio of flip-flops to LUT indicates, Brutus essentially contains a big piece of combinational logic, controlled by a finite state machine (FSM) for the search with 54 states. This probably is one of the most complicated combinational logics ever implemented in an FPGA.

At the top of the module hierarchy, we have our main module, which consists of some statements concerning the communication logic with the PCI-bus, the module Search(), and a logic for clocks. Brutus has 2 clocks with the same frequency, but half a cycle shifted. The additional half a cycle shifted clock is e.g. useful when we write data to block-RAMs, which is a 4096-bit structure in the FPGA. We can only store into such RAM at positive clock edges. By the help of our shifted clock, we can already read the result at the output half a cycle later.

```
module Search();
  ControlLogic;
    // Enables/disables the modules and converts data to the correct
    // format. E.g. the module Board does not know whether
    // MakeMove or UnMakeMove is done. The input is always
    // piece-from, piece-to. The ControlLogic converts the move
    // information in the correct format for the Board module.
  AlphaBetaFSM; // Alphabeta search as Finite State Machine.
    // The FSM sets signals like "MakeMove", "UnMakeMove",
    // "Next-Victim". The next state is determined from the
    // output of the modules and the current state. The signals are
    // used by the ControlLogic to control the operation of the
    // modules. The FSM has 54-States, but some states are simply
    // wait states. It changes no signals. Only a transition to
    // the next state can be done.
  CastleStatus();   // Determine castle status          (0.5 cycles)
  GenVictim();      // Generates next to-square.        (2.0 cycles)
  GenAggressor(); // From-square for given to-square.(2.0 cycles)
  TestCheck();      // Tests if the king is in check.   (2.0 cycles)
  Board();          // Board representation and update (1.0 cycles)
  SearchStack();    // Stack for the search control.    (0.5 cycles)
  Evaluate();       // Evaluation.                      (3.0 cycles)
endmodule

module Evaluate();
  HandleSpecialCases; // E.g: Is there insufficient mate material?
  WeightedSum; // Different weights for different game phases.
  GamePhase(); // Determines the game phase.
  PieceValues(); // Sums up the material value.
  FirstOrderValues(); // piece square tables
  PawnStructure(); // pawn structure and passed pawns
  KingCover(); // pawn-cover around the king
  Dynamic(); // Main part. This evaluation bases on attack and
    // defense relations. E.g. attack on opponents king, mobility
endmodule
```

3.1.1 The Evaluation

The Brutus evaluation consists of many small features which are summed up by the help of one large adder tree. All features are simultaneously determined. In order to show the philosophy behind it, we present the smallest possible feature, i.e. the evaluation for the center control. The module CenterControl gets some input about the pawn positions and attack information on the center squares, which is calculated by some kind of modified move generator. BitCnt6 is a basic module which counts the number of set bits (parallel adder of 6 1-bit variables). We calculate how many times white, and how many times black controls a center square. Then, the side with more attacks on the specific square controls it. In order to make that clearer, we present a small source code example. The equivalent block diagram is shown in figure 3.

```
wire [2:0] wCenterCntE4; //declaration: 3-bit white attack counter.
                  //– sum up the white control on e4
BitCnt6 wCenterCntE4_1(.Bts({wPD3, wPF3
                  // white pawns on d3 and f3 control e4
  wPAttE4,wNAttE4,wBAttE4,wRAttE4}),
                  // white night, bishop and rook attack e4
  .Cnt(wCenterCntE4));
```

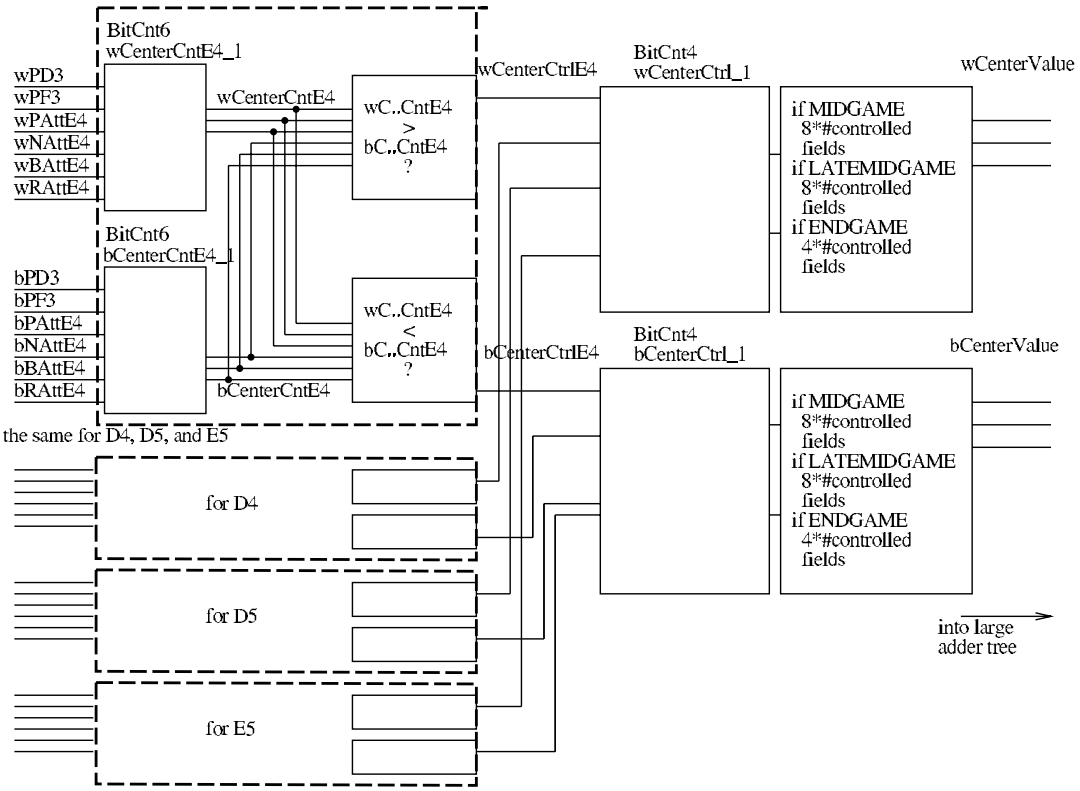

Figure 3. Center Control

```
                    // output: amount of white attacks.
wire [2:0] bCenterCntE4; // the same for black.
BitCnt6 bCenterCntE4_1(.Bts({bPD5,bPF5,bPCapE4,
  bNAttE4,bBAttE4,bRAttE4}),.Cnt(bCenterCntE4));

// White controls e4 if more white than black attacks are on e4.
wire wCenterCtrlE4=(wCenterCntE4¿bCenterCntE4);
                    // The result is 1-bit variable.
// Black Controls e4 if more black than white attacks are on e4.
wire bCenterCtrlE4=(bCenterCntD4¿wCenterCntD4);
```

The same is done for the squares D4,D5,E5 (omitted). Now we count the number of controlled center squares. Here, BitCnt4 is a parallel 4-bit adder.

```
wire [2:0] wCenterCtrlCnt;
BitCnt4 wCenterCtrlCnt_1(.Bts({wCenterCtrlD4,wCenterCtrlE4,
    wCenterCtrlD5,wCenterCtrlE5}),.Cnt(wCenterCtrlCnt));
wire [2:0] bCenterCtrlCnt; // and: the same for black
BitCnt4 bCenterCtrlCnt_1(.Bts({bCenterCtrlD4,bCenterCtrlE4,
    bCenterCtrlD5,bCenterCtrlE5}),.Cnt(bCenterCtrlCnt));
```

We continue by adjusting the output to the present game phase (called 'Phase'). Center control is important in the Mid-Game (MGM), less important in the late mid-game (LATEMGM) and not important at all in the endgame. The value is shifted left by 1 position in mid-game, unchanged in late mid-game, and set to zero in the endgame. The wCenterValue is an input to the white adder tree.

```
wire [3:0] wCenterValue=(Phase==MGM)? wCenterCtrlCnt,1'b0
    :(Phase==LATEMGM)?1'b0,wCenterCtrlCnt:3'h0;
```

3.1.2 The Move Generator

The move generator is, in principle, an 8×8 chessboard. The so called GenAggressor module and the GenVictim module, both instantiate 64 square modules, one for each chessboard square. The GenAggressor and the GenVictim modules determine to which neighbour square incoming signals have to be forwarded. The squares send piece-signals (if there exists a piece on them) resp. forward the signals of the far-reaching pieces. If e.g. the D4-square outputs the signal 'white-rook-up', the victim component will give this to the inputs of D5. Additionally, each square can output the signal 'victim found'. If 'victim found'is sent (i.e. the output bit or output wire is set to 1), we know that this square is the 'victim' (i.e. a to-square) of a legal move. The collection of all 'victim found' signals is the input for an arbiter (indeed a comparator tree), which selects the most attractive, not yet examined victim. Attractivity

of a victim is measured as follows. Most attractive are taking moves, the higher the taken piece the better. If there is no taking move, a move from a 'killer' victim is preferred, and thereafter all other moves are considered according their victim's field address. The GenAggressor module takes the arbiter's output as input and sends the signal of a super-piece (a combination of all possible pieces). If e.g. the rook-move signal hits a rook of our own, we will find an 'aggressor' (i.e. a from-square of a legal move). Again, an arbiter selects the most attractive square which has not yet been used. Thus, from a more abstract point of view, several legal moves are generated in parallel. These moves must be sorted and we have to mask, which of them have already been examined. We sort the moves by the help of a comparator tree. The winner is determined within 2 passes, 6 tree-levels each. Sorting criteria are the values of attacked pieces and whether or not a move is a killer move. We refer to [7] for further details of the principles concerning the move generator and the GenVictim and GenAggressor modules.

3.1.3 The FPGA Search

Figure 4 shows a simplified version of the finite state machine that controls the search on the FPGA card. States must not be interpreted as time points when functions are called. E.g. the move generator and the evaluation are never called in that sense. They are running all the time. If e.g. the next state is QS_AGGR, this has nothing to do with the generation of a move. It just means that we can now acquire the input of the aggressor-module ('aggressor found?', Fig. 4).

This is the big difference between software and hardware. In a hardware design nobody 'calls' anything. All logic units run simultaneously. You can only change the input or acquire the output of a module. In our program, we change the input for the move generator and the evaluation module within the state transitions MakeMove resp. UnMakeMove. In hardware, you must act at the right time, i.e. you need to make sure that all signals are there at the right time. E.g., the current move is on the input of the board-module all the time. The art is to determine the cycle, when the board can be updated, i.e. to synchronize the modules. Otherwise the board module and the evaluation module would work on undefined inputs.

The search works as follows. We enter the search at FS_INIT. If there is anything to do, and if a nullmove is not applicable, we reach the start of the full search, i.e. not the quiescence search. After possibly increasing the search depth (not shown in figure 4), we enter the state FS_VICTIM. Here, the output of the GenVictim module is inspected, and if we find a to-square of a valid move and a futility cutoff is not possible, we will reach the state FS_AGGR. The GenAggressor output is inspected, and if we find a from-square, we will make the next move and come to FS_DOWN. This corresponds to a recursive call of the Alphabeta algorithm. Indeed, it is a variant of the Alphabeta algorithm, which uses only the window $[\alpha, \alpha + 1]$. If the search remaining depth is greater than 0, we start with looking for a move in the state FS_START. Otherwise we enter the quiescence search, which starts with the examination of the evaluation output. If the evaluation is not greater than alpha, we continue with a capture move, if available. If there is a piece which can be taken, we reach the QS_AGGR state, and if we additionally get a from-square by the help of the GenAggressor module, we will make a further move etc. Moves are unmade and we leave a recursion level, whenever we reach the state QS_RETURN. In order to save the necessary internal data in the FPGA when doing a recursive descent in the look ahead tree, we use a stack which is saved in the Block-RAMs of the FPGA.

3.2 The Distributed Search Algorithm

3.2.1 General Framework

The basic idea of our parallelization is to decompose the search tree, to search parts of the search tree in parallel and to balance the load dynamically by the help of the work stealing concept. This works as follows: First, a special processor P_0 gets the search problem and starts performing the negascout algorithm as if it would act sequentially. At the same time, the other processors send requests for work to other randomly chosen processors. When a processor P_i that is already supplied with work, catches such a request, it checks, whether or not there are unexplored parts of its search tree, ready for evaluation. These unexplored parts are all rooted at the right siblings of the nodes of $P_i's$ search stack. Either, P_i sends back that it cannot serve with work, or it sends such a node (a chess position with bounds etc.) to the requesting processor P_j. Thus, P_i becomes a master itself, and P_j starts a sequential search on its own. The processors can be master and worker at the same time. The relationship is dynamically changed during the computation. When P_j has finished its work (possibly by the help of other processors), it sends an answer message to P_i. The master-worker relationship between P_i and P_j is released, and P_j is idle again. It again starts sending requests for work into the network. When a processor P_i finds out that it has sent a wrong window to one of its workers P_j, it makes a window message follow to P_j. P_j stops its search, corrects the window and starts its old search from the beginning. If the message contained a cutoff, P_j just stops its work. A processor P_i can shorten its search stack from outside, when e.g. a cutoff message comes in which belongs to a node near the root.

In distributed systems, messages can be delayed by the

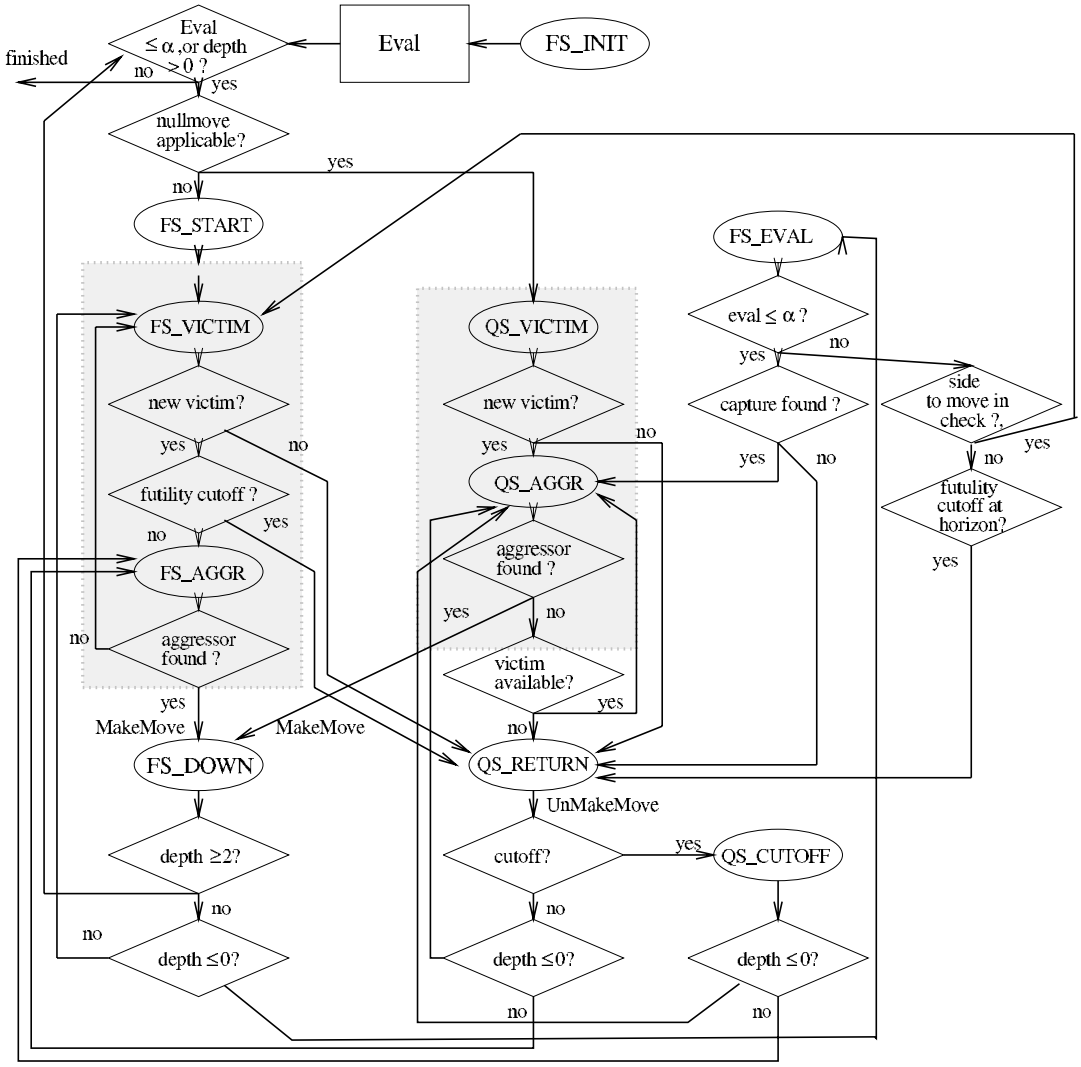

Figure 4. Flowchart for the Finite State Machine for the Search.

system. Messages from the past might arrive, which are outdated. Therefore, each message gets its own unique ID, such that we are always able to identify misleading results. We refer to [5] for further details.

3.2.2 Selection of Subproblems

The distributed subproblems are supposed to be large enough to keep the workers busy for a while. Otherwise, the communication overhead increases, since at least two messages (the subproblem itself and the result message) have to be exchanged for every subproblem. We can control the granularity of the subproblems by the minimum remaining search depth for each subproblem. We only distribute subproblems with a remaining search depth of at least 6 plies. In order to avoid search overhead, it is often useful to delay the use of parallelism, such that superfluous coordination effort can be kept small. Therefore, we use a variant of the Young Brother Wait Scheduling (YBWC). YBWS states that the parallel evaluation of a right ('younger') sibling of a present search node may start only after the evaluation of the leftmost sibling has been finished. By the use of the YBWS the parallel search performs all cutoffs produced by the result of the evaluation of the leftmost sons. Sometimes it is even better to delay parallelism even more until not only the leftmost, but also its right sibling, have been finished. In Brutus, this is dynamically controlled. When the work load is low, parallelism starts as soon as the first successor is completed. Otherwise two successors have to be completed.

3.2.3 Distributed Hashtables

A distributed hashtable is one of the most sensitive, but also most important features of parallel game tree search. In such a hashtable, all former results — values as well as best moves and cutoff moves – are stored. This dramatically improves the ordering of the moves, especially when iterative deepening is used. We have developed a hierarchical hashtable system. All processes which are coupled by shared memory, use only one hashtable. It is not synchronized at all. With up to four shared memory machines, we collect hashtable puts in so called baskets and broadcast them when the baskets are filled. Even larger networks could not be tested yet, but we investigated simulations with up to 16 processors. 'Simulation' means that we concentrated on our software search, using only a simple quiescence search with pure piece value information instead of depth-3 FPGA searches. We made experiments with an asynchronous distributed hashtable mechanism. The hashtable is partitioned into $n = N/4$ many partitions, N being the number of shared memory machines. Each processor keeps n baskets and hash entries are collected therein. When a basket is complete, it is sent to the corresponding sub-cluster and distributed there. A hash-inquiry is asynchronous as well: In order to avoid waiting-times on a busy processor P_i, P_i does not wait for the hash-answer, but it just proceeds as if there were no hash entry for the present node. There are two possibilities: a) no hash answer arrives, then P_i acts in exactly the right way, or b) a hash-answer arrives a bit later. Then P_i stops its search and restarts the search below the node which belongs to the hash answer. In the latter case, it has wasted some time. Nevertheless, the idea is that near leaves, mostly there is no hash entry, and near the root it does not matter, whether or not P_i wasted some microseconds. This seems properly work in the sense that we measured speedups of about 14 on 16 processors, but real experiments are still open.

4 Results

Experiments with Brutus are performed on a cluster of the Paderborn University. Each processor is a Pentium IV/2.8 GHz running the RedHat Linux operating system. The processors are connected by a Myrinet high speed interconnection network. Each node of the network consists of a dual computer. With our small number of processors, we have never seen problems with inter-node latencies. Each processor is supplied with its own Alphadata VirtexV1000E FPGA card.

We measure speedups of our program by the help of the BT2630 test set (30 positions). The procedure is as follows. The parallel 4 processor version is allowed to examine each position of the test set for a certain period of time. The time period is chosen according to chess tournament rules. Then we look up, how many iterations of the iterative deepening have been performed and then the sequential version and the 2 processor version have to reach the same searching depth on each test position. We compare the running times and the number of used search nodes. The speedup (SPE) is the sum of the times of the sequential version divided by the sum of the times of a parallel version. Concerning [6] this is the right way for measuring speedups. The search overhead is given in percent (work %) seen from 100% of the sequential version. Thus, work%=-10 means that the parallel version used 10 percent less nodes than the sequential version does.

	time(s)	SPE	work %
1	19302	1	0
2	9419	2.04	-10
4	5016	3.85	-7

Brutus' Speedup Results on the BT2630 Test

Brutus' playing strength has grown rapidly. Started in October 2000, it reached a third rank on the 10^{th} World Computer Chess Championship 2002 in Maastricht, the third rank on the Paderborn Computer Chess Championship in February 2003 and recently, it has won the Lippstadt FIDE Grandmaster Tournament with 9 out 11 against opponents with an average ELO of 2506. Thus, it has reached a tournament performance of 2768 ELO points (The ELO-system is a statistical measure for the strength of chess players. An International Master has about 2450 ELO, a Grandmaster about 2550, the top-ten chess players have about 2700 ELO on the average, and the human World Champion about 2800 ELO.[2].) When we take into consideration that one of the draws was caused by technical problems, and one of the draws was a fair-play draw, we can estimate Brutus' playing strength to be far more than 2800 ELO. Internal test matches support this conjecture: 140 test games against Fritz8, playing on a Pentium 2,4 GHz PC, showed an advantage of 113 ELO points. Fritz8 running on an Athlon 1,2 GHz, however, has 2762 ELO, concerning the Swedish Rating List.

5 Conclusion and Future Work

We presented the professional chess program Brutus, which is one of the leading chess programs in the world. Brutus is the first chess program which uses the FPGA technology, and it is the first one which combines the hardware design methods of Deep Blue with fully dynamic game tree

[2] typically, USCF numbers are about 100 points higher than ELO numbers

search approaches as described in [5]. The main advantage of the hardware design is that the implementation of further chess knowledge does not result in a slower execution of the program. The program just needs more area on its chip. The FPGA technology seems to be the right compromise, as on the one hand it is possible to make use of the fine grained parallelism, as it is common in hardware designs. On the other hand we have no long development cycles, which are also typical for hardware components. In computer chess it is essential to have the possibility to frequently change and improve the program's code, since development cycles must be kept extremely short.

References

[1] A. de Bruin A. Plaat, J. Schaeffer and W. Pijls. A minimax Algorithm better than SSS*. *Artificial Intelligence*, 87:255–293, 1999.

[2] T.S. Anantharaman. Extension heuristics. *ICCA Journal*, 14(2):47–63, 1991.

[3] J.H. Condon and K. Thompson. Belle chess hardware. *Advances in Computer Chess III, M.R.B. Clarke (Editor), Pergamon Press*, pages 44–54, 1982.

[4] C. Donninger. Null move and deep search. *ICCA Journal*, 16(3):137–143, 1993.

[5] R. Feldmann, M. Mysliwietz, and B. Monien. Studying overheads in massively parallel min/max-tree evaluation. In *6th ACM Annual symposium on parallel algorithms and architectures (SPAA'94)*, pages 94–104, New York, NY, 1994. ACM.

[6] P.J. Flemming and J.J. Wallace. How not to lie with statistics: the correct way to summerize benchmark results. *CACM*, 29(3):218–221, 1986.

[7] F-H. Hsu. Ibm's deep blue chess grandmaster chips. *IEEE Micro*, 19(2):70–80, 1999.

[8] F-H. Hsu, T. S. Anantharaman, M.S. Campbell, and A. Nowatzyk. *Computers, Chess, and Cognition*, chapter 5 Deep Thought, pages 55–78. Springer Verlag, 1990.

[9] R.M. Hyatt, B.E. Gower, and H.L. Nelson. Cray blitz. *Advances in Computer Chess IV, D.F. Beal (Editor), Pergamon Press*, pages 8–18, 1985.

[10] D.E. Knuth and R.W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975.

[11] U. Lorenz. Controlled Conspiracy-2 Search. *Proceedings of the 17th Annual Symposium on Theoretical Aspects of Computer Science (STACS), (H. Reichel, S.Tison eds), Springer LNCS*, pages 466–478, 2000.

[12] D.A. McAllester. Conspiracy Numbers for Min-Max searching. *Artificial Intelligence*, 35(1):287–310, 1988.

[13] A. Reinefeld. *Spielbaum - Suchverfahren*. Springer, 1989.

[14] J. Schaeffer. Conspiracy numbers. *Artificial Intelligence*, 43(1):67–84, 1990.

[15] C.E. Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 41:256–275, 1950.

[16] D.J. Slate and L.R. Atkin. Chess 4.5 - the northwestern university chess program. *Chess Skill in Man and Machine, P.W. Frey (Editor), Springer Verlag*, pages 82–118, 1977.

[17] K. Thompson. personal communication. 2000.

0-7695-2132-0/04/$17.00 (C) 2004 IEEE